Épreuve d'informatique

I – Un jeu d'argent

On considère le jeu suivant.

Chacun des deux joueurs met successivement sur la table une quantité d'argent égale à un, deux, ou cinq euros. Le joueur qui gagne est celui qui ajoute de l'argent de façon que la somme totale atteigne un nombre au carré, soit 4, 9, etc. ou atteigne un nombre maximal, que l'on prend ici égal à 9. Le jeu s'arrête donc lorsque la somme d'argent atteint 4 ou 9. Un joueur n'est pas autorisé à dépasser 9 lors de sa mise.

Modélisation du jeu — Nous commençons par modéliser le jeu avec les outils des graphes.

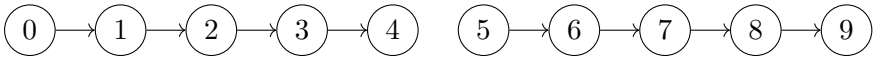
1) Quels adjectifs parmi les suivants s'appliquent à ce jeu : coopératif, non coopératif, simultané, séquentiel, à information totale, à information partielle, jeu d'accessibilité ? Justifier.

Solution

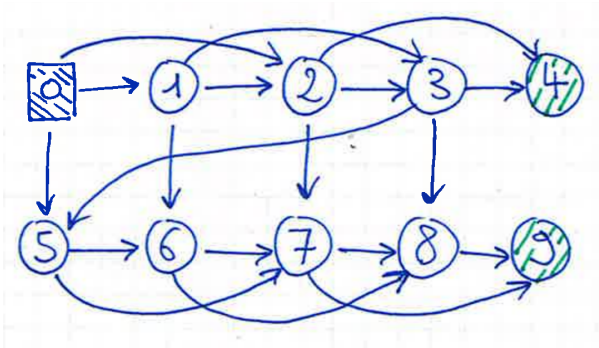
Jeu non coopératif, séquentiel, à information totale, d'accessibilité.

2) On représente ci-dessous le graphe orienté du jeu, noté G, où seuls les arcs correspondant à l'ajout d'un euro ont été représentés. Les sommets sont étiquetés par la somme d'argent sur la table. Ajouter les arcs correspondant à l'ajout de deux et cinq euros.

On tachera de rendre le graphe le plus lisible possible.



Solution



3) Définir l'origine s_0 du graphe et l'ensemble des sommets finaux F et les colorer sur le graphe précédent. Définir l'ensemble des sommets S et l'ensemble des arcs A. Ce graphe G est-il biparti ?

Solution

L'origine est $\boxed{s_0 = 0}$.

L'ensemble des sommets finaux est $\boxed{F = \{4, 9\}}$.

L'ensemble des sommets est $\boxed{S = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}}$.

L'ensemble des arcs est

$$\boxed{\begin{aligned} A = \{&(0,1),(0,2),(0,5),(1,2),(1,3),(1,6),(2,3),(2,4),(2,7),(3,4),(3,5),(3,8),(5,6),(5,7),\\ &(6,7),(6,8),(7,8),(7,9),(8,9)\}. \end{aligned}}$$

Le graphe G n'est $\boxed{\text{pas biparti}}$ *car il existe des cycles (non orientés) impairs, par exemple le cycle* $0 \to 1 \to 2 \leftarrow 0$, *ce qui interdit d'avoir une bipartition* $S = S_1 \cup S_2$ *tels que les arcs relient uniquement les sommets de* S_1 *avec ceux de* S_2.

4) Implémenter le graphe G dans le langage `python` sous forme d'une liste d'adjacence, représentée par un dictionnaire `G`. On rappelle la syntaxe d'un dictionnaire : `{clé1:valeur1,clé2:valeur2,...}`

Solution

```
G = {0:[1,2,5],1:[2,3,6],2:[3,4,7],3:[4,5,8],4:[],5:[6,7],6:[7,8],7:[8,9],8:[9],9:[]}
```

5) Quelle propriété possède l'origine s_0 vis-a-vis du dictionnaire `G` ? Compléter les parties pontillés du code suivant pour définir la fonction `origine` qui prend en argument le dictionnaire `G` et renvoie l'origine du graphe. On pourra utiliser la méthode `liste.remove(element)` qui retire `element` de `liste`. Commenter toutes les lignes du code. *Attention : la commande `liste.remove(element)` renvoie une erreur si `element` n'est pas dans `liste` !*

```python
def origine(G):
    """
    Entrée : G (dict)
    Sortie : origine de G (int)
    """
    S = G.keys()
    for s in S:
        for ... in ...:
            ...
    return S[0]
```

Solution

L'origine est sans prédécesseurs, donc $\forall s \in S, s_0 \notin G[s]$. On en déduit la fonction `python` suivante.

```python
def origine(G):
    """
    Entrée : G, un dictionnaire représentant un graphe orienté
    Sortie : s0, l'origine de G
    """
    S = G.keys()          # Liste des sommets du graphe
    for s in S:           # Parcours des sommets du graphe
        for p in G[s]:    # Si p est un successeur de s
            S.remove(p)   # On retire p de la liste des sommets
    return S[0]           # On retourne l'origine de G
```

6) Effectuer l'exécution pas-à-pas de l'appel `origine(G)` sur le dictionnaire `G` défini précédemment, en indiquant la valeur initiale de la liste `S`, puis la valeur des variables `s`, `G[s]`, et `S` à l'issue de chaque itération dans la boucle, et enfin la valeur de `S[0]` retournée par la fonction.

Solution

La valeur initiale de `S` est `[0,1,2,3,4,5,6,7,8,9]`.

s	G[s]	S	S[0]
0	[1,2,5]	[0,3,4,6,7,8,9]	0
1	[2,3,6]	[0,4,7,8,9]	0
2	[3,4,7]	[0,8,9]	0
3	[4,5,8]	[0,9]	0
4	[]	[0,9]	0
5	[6,7]	[0,9]	0
6	[7,8]	[0,9]	0
7	[8,9]	[0]	0
8	[9]	[0]	0
9	[]	[0]	0

La fonction renvoie alors

$$\boxed{S[0] = 0}$$

Remarque

7) Quelle propriété possède les sommets finaux vis-a-vis du dictionnaire `G` ? Implémenter une fonction `final` qui prend en argument le dictionnaire `G` et renvoie la liste `F` des sommets finaux. Les spécifications de la fonction seront donnés et le code sera commenté.

Solution

Les sommets finaux sont sans successeurs, soit $G[s] = [\,], \forall s \in F$. On en déduit la fonction `python` suivante.

```python
def final(G):
"""
Entrée : G, un dictionnaire représentant un graphe orienté
Sortie : F, une liste des sommets finaux
"""
F = []              # Initialisation de la liste des sommets finaux
for s in G:         # Parcours des sommets du graphe
    if G[s] == []:  # Si le sommet s n'a pas de successeur
        F.append(s)   # On ajoute s à la liste des sommets finaux
return F            # On retourne la liste des sommets finaux
```

Proposition de stratégies — Dans la suite, on ne représentera pas le graphe biparti associé à G mais on gardera à l'esprit qu'un même sommet $0 \leqslant i \leqslant 9$ peut représenter deux configurations différentes du jeu suivant que le sommet appartient au premier joueur J_1 ou au second joueur J_2.

8) Déterminer quels sommets peuvent appartenir à J_1 ou J_2 et définir les ensembles correspondant S_1 et S_2. Attention, certains sommets n'appartiennent qu'à un seul des joueurs, tandis que d'autres peuvent appartenir aux deux joueurs.

Solution

$\boxed{S_1 = \{0,2,3,4,5,6,7,8,9\}}$ *et* $\boxed{S_2 = \{1,2,3,4,5,6,7,8,9\}}$. *Le sommet* 0 *n'appartient qu'à* J_1 *tandis que* 1 *n'appartient qu'à* J_2.

9) On définit la fonction $f : S_1 \setminus F \to S_2$ telle que $f(0) = 5$, $f(2) = 4$, $f(3) = 4$, $f(5) = 7$, $f(6) = 8$, $f(7) = 9$, et $f(8) = 9$. Cette fonction définit-elle une stratégie ? Si oui, cette stratégie est-elle gagnante pour le joueur J_1 ? Justifier.

Solution

L'ensemble de départ est bien défini sur S_1/F *car* $1 \notin S_1$ *et* $F = \{4,9\}$. *L'ensemble d'arrivée est bien défini car* $\{1,5,4,8,9\} \cap S_2$.

La fonction f *définit bien une* $\boxed{\text{stratégie}}$ *car les paires* $(0,1)$, $(2,4)$, $(3,4)$, $(5,7)$, $(6,8)$, $(7,9)$, *et* $(8,9)$ *sont bien des arcs.*

La stratégie f *n'est* $\boxed{\text{pas gagnante}}$ *pour le joueur* J_1. *En effet, la partie* $0 \xrightarrow{J_1} 5 \xrightarrow{J_2} 6 \xrightarrow{J_1} 8 \xrightarrow{J_2} 9$ *est jouée suivant* f *mais perdante pour* J_1 *car le joueur* J_2 *passe de* 6 *au sommet final* 9.

10) On définit la stratégie $g : S_2 \setminus F \mapsto S_1$ telle que $g(1) = 6$, $g(2) = 4$, $g(3) = 4$, $g(5) = 6$, $g(6) = 7$, $g(7) = 9$, et $g(8) = 9$. Montrer que cette stratégie est gagnante pour le second joueur J_2.

Solution

La stratégie g *est* $\boxed{\text{gagnante}}$ *pour le joueur* J_2. *En effet, toutes les parties jouées suivant* g *sont de la forme* $0 \xrightarrow{J_1} 1,2,5 \xrightarrow{J_2} 4,6(\xrightarrow{J_1} 7,8 \xrightarrow{J_2} 9)$ *et sont gagnantes pour* J_2.

Algorithme min-max — Pour déterminer de façon certaine s'il existe ou non une stratégie gagnante pour l'un ou l'autre des joueurs, nous appliquons l'algorithme min-max.

11) Rappeler le principe de l'algorithme min-max. Rappeler l'avantage de la représentation en arbre par rapport à la représentation en graphe.

Solution

Voir cours. Un graphe représente toutes les configurations du jeu possibles ; une arbre représente toutes les parties possibles.

12) Représenter l'arbre de décision associé à G en indiquant à quel joueur appartiennent les sommets de chaque étage de l'arbre, puis appliquer l'algorithme min-max à la main sur l'arbre de décision. En déduire s'il existe une stratégie gagnante pour J_1 ou pour J_2.

13) On implémente la fonction `minmax` suivante. Compléter les parties pointillées du code en utilisant les méthodes `max` et `min` de python, et commenter chaque ligne du code.

```python
def minmax(G,s):
    """
```

```
    Entrée : graphe G (dictionnaire), sommet initial s
    Sortie : heuristique h(s) (entier)
    """
    if G[s] == []:
        if s[0] == 1:
            return ...
        else:
            return ...
    else:
        h = [minmax(G,v) for v in G[s]]
        if s[0] == 1:
            return ...
        else:
            return ...
```

Solution

```
def minmax(G,s):
    """
    Entrée : graphe G (dictionnaire), sommet initial s = (j,i)
    Sortie : heuristique h(s) (entier)
    """
    if G[s] == []:
        if s[0] == 1:
            return -1
        else:
            return +1
    else:
        h = [minmax(G,v) for v in G[s]]
        if s[0] == 1:
            return max(h)
        else:
            return min(h)
```

Algorithme du noyau — On souhaite vérifier la conclusion précédente par un l'algorithme du noyau. Le *noyau* d'un graphe orienté G est le sous-ensemble de sommets $N \subset S$ tel que :

(i) aucun arc ne joint deux sommets de N : $\forall s_1, s_2 \in N, (s_1, s_2) \notin A$,

(ii) tout sommet hors de N a au moins un successeur dans N : $\forall s_1 \in S \setminus N, \exists s_2 \in N, (s_1, s_2) \in A$.

14) On peut construire le noyau récursivement. Soit G un graphe dont l'ensemble des sommets finaux est F. On initialise le noyau par l'ensemble vide puis on procède de la manière suivante :

a) on choisit un sommet final $s \in F$, que l'on ajoute au noyau N ;

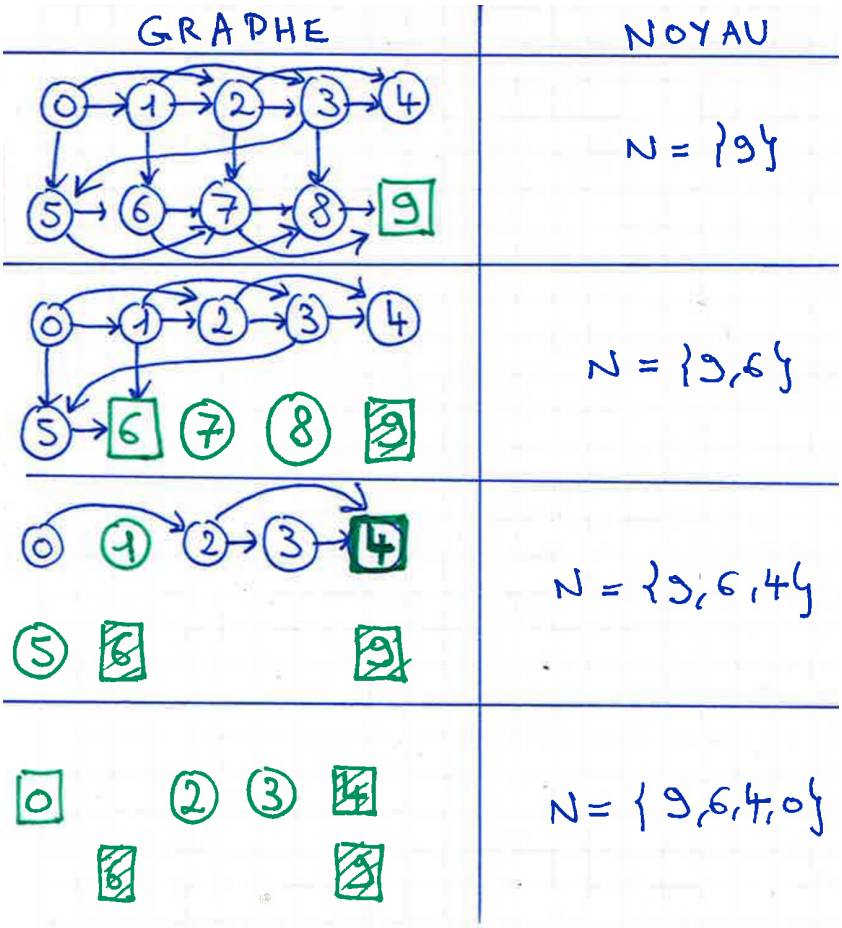
b) on retire de G tous les prédécesseurs p de s, ainsi que tous les arcs dont p ou s est une extrémité ;

c) on répète le processus sur le sous-graphe obtenu jusqu'à ce que le sous-graphe soit vide.

Appliquer cet algorithme sur le graphe G précédent à partir du sommet final $s = 9$ en représentant à chaque étape le sous-graphe obtenu et le contenu du noyau. Montrer que le noyau du graphe G est l'ensemble $N = \{0, 4, 6, 9\}$.

Solution

Lors de la deuxième étape, le sous-graphe contient deux sommets finaux : 4 et 6. On peut choisir l'un et l'autre de ces sommets pour continuer la procédure.



15) On admet que le noyau N contient l'ensemble des *positions perdantes pour* J_1. Le complémentaire du noyau dans S donne au contraire les *positions gagnantes pour* J_1. Conclure sur l'existence d'une stratégie gagnante pour J_1 ou J_2.

Solution

Il existe une stratégie gagnante pour J_2 mais pas pour J_1.